MyCodeSchool

teorie — OOP în TypeScript

clasă · încapsulare · moștenire · polimorfism · abstractizare · pe POM-urile din `sap-starter` · Andreea

Comandă	Ce face
`npx tsx src/teorie/A1-incapsulare-private/main.ts`	rulează un fișier (ignoră tipurile)
`npx tsc --noEmit`	verifică tipurile în tot proiectul: **poarta**; zero erori = gata
`npx playwright test --project=chromium`	suita din `sap-starter`: cifrele trebuie să rămână aceleași

1. Clasa — anatomie (demo A2)

```
class Procent {
  readonly nume: string;
  private _valoare = 0;
  constructor(nume: string) {
    this.nume = nume;
  }
  get valoare(): number {
    return this._valoare;
  }
  set valoare(noua: number) {
    this._valoare =
      Math.min(100, Math.max(0, noua));
  }
}
const acoperire = new Procent("Acoperire");
acoperire.valoare = 140;
```

Sintaxă	Ce face
`class X { }`	șablonul
`new X(...)`	un obiect nou; rulează `constructor`
`this.nume`	câmpul obiectului curent
`readonly`	scris doar la declarare sau în constructor
`private _valoare`	doar codul din clasă îl vede
`get valoare()`	`o.valoare` = apel, arată ca un câmp
`set valoare(v)`	`o.valoare = v` trece prin metodă
`get` fără `set`	doar citire (`esteSetat`)
`throw new Error()`	refuză o valoare greșită
`async` + `Promise<T>`	metodă cu `await`, întoarce un `T`

```
acoperire.valoare = 140
        │
        ▼
set valoare(140) ─► _valoare = 100
                          │
acoperire.valoare ◄───────┘    100
```

2. Încapsulare — cine vede ce câmp

Modificator	în clasă	în subclasă	în `main` / spec
`private`	da	nu	nu
`protected`	da	da	nu
`public`*	da	da	da
`readonly`	se citește ca mai sus, nu se mai scrie · *implicit		

Regula exercițiilor: câmpurile sunt `private` sau `protected`; din `main` doar metode și accesori.

```
class Numarator
  protected valoare ◄── NumaratorCuLimita: da
                    ◄── NumaratorCuPas:    da
                    ✗── main: n.valoare → TS2445

class ListaDeRanduri
  protected randuri ◄── ListaFaraDuplicate
  private   → subclasa nu-l mai vede: TS2341
```

3. Moștenire — ierarhia paginilor din `sap-starter`

```
SapTableBrowserPage      page, commandInput, btnOpen, btnExecute,
   ▲                     resultCount, goTo(), filterInput(k), cell(r, c),
   │ extends             getCellText(r, c), getRowCount(), executeTable(...)
   │
TablePage  «abstract»    abstract tableName, abstract columns,
   ▲                     open(), readColumnNames(), tableCell(r, c)
   │ extends
   ├── FrParentPage      tableName, columns, filterESID ..., readRow()
   ├── ItmDetailsPage    tableName, columns, filterPARENTID ..., readRow()
   ├── MailCountPage     ├── EntitlementPage
   └── FrAuditPage       └── CaseNotesPage     (doar tableName, columns,
                                                filtrele proprii)
```

```
new FrParentPage(page)
 └► constructor FrParentPage(page)
     ├ 1 super(page)
     │    └► TablePage: n-are constructor
     │         └► SapTableBrowserPage:
     │              this.page, btnExecute, ...
     └ 2 this.filterESID =
          this.filterInput("ESID")
          ▲ abia după super
```

Sintaxă	Ce face
`class B extends A`	B are tot ce are A; B **este un** A
`super(...)`	constructorul bazei; prima linie, înainte de orice `this.`
fără constructor în B	îl moștenește pe al lui A
metodă cu același nume	override: B o înlocuiește
`super.metoda()`	varianta din A, chemată din override
override fără `super.adauga()`	regulile bazei dispar, în tăcere
ordinea lui `super.m()`	se vede în rezultat: care eroare iese prima
etaj 3: `C extends B`	`NumaratorCiclic` → `CuLimita` → `Numarator`

```
class InregistrareEroare
    extends Inregistrare {
  readonly cod: number;
  constructor(m: string, t: string,
              cod: number) {
    super(m, t);
    this.cod = cod;
  }
  formateaza(): string {
    return `${super.formateaza()} (cod ${this.cod})`;
  }
}
```

clasa din mijloc	codul comun a 2 surori urcă într-o bază (`ExportatorCuSeparator`)
câmp `readonly primul: Raport`	compoziție: **are un** (`RaportCompus`), nu `extends`

4. Polimorfism — o buclă, fiecare obiect răspunde în felul lui

```
const tablePages: TablePage[] = [frParentPage, itmDetailsPage, ...,
                                 caseNotesPage];
for (const tablePage of tablePages) {
  await tablePage.open();  ──► this.tableName = ?
}                                  │
  frParentPage   — obiectul real ──┼──► "/CFF/FR_PARENT"
  mailCountPage  — obiectul real ──┼──► "/CFF/MAILCOUNT"
  caseNotesPage  — obiectul real ──┘──► "/CFF/CASE_NOTES"
  tabel nou = clasă nouă + o linie în array; bucla nu se atinge
```

```
const coloane: Coloana[] = [
  new Coloana("ESID"),
  new ColoanaData("CREATED_ON"),
  new ColoanaSuma("AMOUNT"),
];
const rand = ["ESID-100", "2026-10-02",
              "1234.5"];
for (let i = 0; i < coloane.length; i++) {
  console.log(coloane[i].afiseaza(rand[i]));
}
```

```
ESID-100
02.10.2026
1234.50 RON
```

Ce scrii	Ce decide
tipul declarat `: Raport`, `: TablePage[]`	ce metode **ai voie** să chemi
obiectul din `new RaportSemnat(...)`	care `text()` **rulează**
parametru `v: Verificare[]`	primește orice subclasă, și pe cele de mâine
`p.readRow()` pe `p: TablePage`	refuzat: `readRow` e doar în `FrParentPage`
`class Negata` cu câmp `interior: Verificare`	un obiect care întoarce opusul altuia

În buclă: niciun `instanceof`, niciun `if / switch` pe tipul sau numele obiectului.

5. Abstractizare — clasa care cere, subclasa care dă

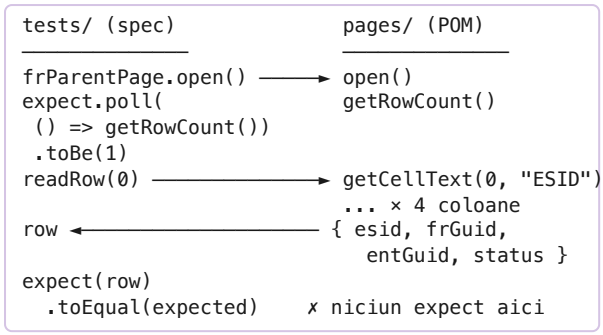
	`abstract class TablePage`	`class FrParentPage`	`interface FrParentRow`
`new`	refuzat: TS2511	da (fixture-ul)	nu: e doar un tip
câmp cerut	`abstract readonly tableName: string;`	`readonly tableName = tables.frParent;`	`esid: string;`
metodă cerută	`abstract corp(): string;` fără corp	obligată s-o scrie, altfel TS2515	`caRanduri(): string[][];` (`Exportabil`)
cod scris	`open()` o singură dată	moștenit	niciunul, doar forma
se leagă cu	`extends`: o singură bază		`implements`: oricâte; lipsă = TS2420
pas ascuns	`protected abstract verificaRand(rand)`	îl scrie, doar baza îl cheamă	—
la ce	o familie de pagini	un tabel concret	forma unui rând; o capacitate

```
frParentPage.open()     scrisă o dată, în TablePage
 1 this.goTo()
 2 this.commandInput.fill(transactions.se16)
 3 this.btnOpen.click()
 4 this.tableNameInput.fill(this.tableName)
                                ▲ dat de subclasă
 5 this.btnExecute.click()
baza fixează pașii; subclasa dă bucata
(Notificare.trimite() ─► corp(),
 Raport.text() ─► antet() + subsol())
```

```
export interface FrParentRow {
  esid: string; frGuid: string;
  entGuid: string; status: string;
}
interface Exportabil {
  caRanduri(): string[][];
}
class Notita implements Exportabil {
  caRanduri(): string[][] {
    return [["nota"]];
  }
}
```

Contract scris de JS	Cine îl cheamă
`toString(): string`	`` `${obiect}` ``, `String(obiect)`
`toJSON()`	`JSON.stringify(obiect)`

6. Regula din curs — pagina citește, spec-ul verifică



`pages/`	`tests/`
locatori, `open()`, filtre	datele așteptate (cazuri)
numele coloanelor SAP	`expect`, `toEqual`, `expect.poll`
întoarce `Promise<FrParentRow>`	compară rânduri, nu celule
zero `expect`	**zero nume de coloană**

```
await expect.poll(
  () => frParentPage.getRowCount()).toBe(1);
expect(await frParentPage.readRow(0))
  .toEqual(testCase.expected);
```

> Test care n-a fost văzut **roșu** nu dovedește nimic: strică o valoare, vezi diff-ul, pune-o la loc.

7. Greșeală → ce spune `npx tsc --noEmit`

Greșeală	Eroarea
`s.randuri = []` în `main`, câmp `private`	`TS2341: Property 'randuri' is private and only accessible within class 'SetDeDate'.`
`n.valoare` în `main`, câmp `protected`	`TS2445: Property 'valoare' is protected and only accessible within class 'Numarator' and its subclasses.`
constructor de subclasă fără `super(...)`	`TS2377: Constructors for derived classes must contain a 'super' call.`
`this.x = ...` înainte de `super(...)`	`TS17009: 'super' must be called before accessing 'this' in the constructor of a derived class.`
scriere în `readonly` sau în `get` fără `set`	`TS2540: Cannot assign to 'schema' because it is a read-only property.`
subclasă fără membrul `abstract`	`TS2515: Non-abstract class 'MailCountPage' does not implement inherited abstract member tableName from class 'TablePage'.`
`new` pe clasa abstractă	`TS2511: Cannot create an instance of an abstract class.`
metodă din subclasă chemată pe tipul bazei	`TS2339: Property 'readRow' does not exist on type 'TablePage'.`
`implements` fără metoda din interfață	`TS2420: Class 'Notita' incorrectly implements interface 'Exportabil'.`

8. Exercițiu → noțiuni

Exercițiu	Noțiuni	Demo · §
`vanilla-ts` 01 `SetDeDate`	clasă, constructor, `private`, `throw new Error`	A1 · §1, §2
02 `NumeTabel`	`get` / `set`, `readonly`, accesor doar de citire	A2 · §1
03 `Raport` · 04 `Numarator`	`extends`, `super(...)`, `super.text()`; `protected`	B1, B2 · §3
05 `Verificari` · 06 `Exportatoare`	override, `Verificare[]`, o buclă; `abstract`, `numeFisier()` în bază	C1, C2 · §4, §5
fișa moștenire 1–7	`super` în constructor și metodă, ordinea lui `super`, etaj 3, compoziție, clasa din mijloc	B1, B2 · §3
fișa polimorfism 1–7	tip nou fără să atingi bucla, parametru de tipul bazei, tip declarat vs obiect, `Map`, `Negata`	C1 · §4
fișa abstractizare 1–7	metodă `abstract`, baza fixează pașii, `protected abstract`, `interface` / `implements`, `toString` / `toJSON`	C2 · §5
SAP 01 moștenire	`TablePage`, 6 clase `extends TablePage`, `super(page)`	§3
SAP 02 abstractizare	`abstract class TablePage extends SapTableBrowserPage`, `abstract readonly tableName`, `open()`	§3, §5
SAP 03 polimorfism	`abstract columns`, `TablePage[]`, `for...of`, `toEqual` pe array	§4
SAP 04 încapsularea datelor	`interface FrParentRow`, `readRow(): Promise<FrParentRow>`, `toEqual` pe obiect	§5, §6